

■ File I/O (Input / Output)

File I/O means **reading data from files** and **writing data to files** using Python's built-in functions.

File Opening Modes — `open("filename", "mode")`

Mode	Meaning	Key detail
"r"	Read	Default. Error if file doesn't exist
"w"	Write	Creates new file OR overwrites existing
"a"	Append	Adds to end; creates file if missing
"r+"	Read + Write	File MUST exist
"w+"	Write + Read	Overwrites existing file
"a+"	Append + Read	Reads + adds to end of file
"b"	Binary	For images/videos — combine: 'rb', 'wb'

■ *Quick decision: just reading? → r | fresh write? → w | adding content? → a*

Reading from a File

Method	What it does
<code>f.read()</code>	Returns ENTIRE file as one string
<code>f.readline()</code>	Returns ONE line at a time (call repeatedly)
<code>f.readlines()</code>	Returns ALL lines as a LIST of strings

```
# read() — whole file at once
f = open("data.txt", "r")
content = f.read()
print(content)
f.close() # always close!

# readline() — one line at a time
f = open("data.txt", "r")
line1 = f.readline() # first line
line2 = f.readline() # second line
f.close()

# readlines() — all lines as list
f = open("data.txt", "r")
lines = f.readlines() # ['line1\n', 'line2\n', ...]
f.close()
```

Writing to a File

Method	What it does
<code>f.write(str)</code>	Writes a single string to the file

🌸 Python Fundamentals 🌸 Part 5

File I/O • Exception Handling • List Comprehensions • JSON

f.writelines(list)	Writes a list of strings all at once
---------------------------	--------------------------------------

```
# write() – single string
f = open("data.txt", "w")
f.write("Hello students!")
f.close()

# writelines() – multiple lines
f = open("data.txt", "a")
f.writelines(["Line 1\n", "Line 2\n"])
f.close()
```

■ Best Practice: with open() — context manager

Automatically closes the file when the block ends — even if an error occurs. Always prefer this over manual open/close.

```
# Reading with context manager
with open("data.txt", "r") as f:
    content = f.read() # file auto-closes after block
print(content)

# Writing with context manager
with open("notes.txt", "w") as f:
    f.write("Never forget to use with!")
```

Deleting a File

```
import os
os.remove("data.txt") # deletes the file
```

■ *Always check if the file exists before deleting: if os.path.exists('file.txt'): os.remove('file.txt')*

■ Exception Handling

An **exception** is a runtime error that crashes your program if unhandled. Exception handling lets you manage errors **gracefully** so your program keeps running.

Common Built-in Exceptions

Exception	Triggered when...	Example
ZeroDivisionError	Divide by zero	10 / 0
NameError	Variable not defined	print(x) # x never set
FileNotFoundError	File doesn't exist	open('missing.txt')
TypeError	Wrong data type	'2' + 2
ValueError	Right type, wrong value	int('hello')

🌸 Python Fundamentals 🌸 Part 5

File I/O • Exception Handling • List Comprehensions • JSON

IndexError	Index out of range	[1,2,3][9]
KeyError	Dict key not found	d['missing']

Full try / except Structure

Block	When it runs
try	Always — this is the code that might raise an error
except ExceptionType	ONLY if the matching exception occurs
else	ONLY if NO exception occurred in try
finally	ALWAYS — error or not (use for cleanup)

```
try:
x = int(input('Enter a number: '))
result = 10 / x
except ZeroDivisionError:
print("Can't divide by zero!")
except ValueError:
print('Please enter a valid number!')
else:
print('Result =', result) # runs only if no error
finally:
print("Done!") # ALWAYS runs
```

■ *Use specific exceptions (ZeroDivisionError) over bare except: — it makes debugging much easier!*

Practical Examples

```
# File handling with exception
try:
with open("data.txt") as f:
print(f.read())
except FileNotFoundError:
print("File not found!")
finally:
print("Execution completed.")
```

■ List Comprehensions

A **short, elegant one-liner** to build lists — replaces verbose for-loops with clean, readable expressions.

Type	Syntax	Use when...
Basic	[expr for item in iterable]	Transform every item
Filtered	[expr for item in iterable if condition]	Keep only matching items
if-else	[t if cond else f for item in iterable]	Assign one of two values per item

Basic — transform every item

🌸 Python Fundamentals 🌸 Part 5

File I/O • Exception Handling • List Comprehensions • JSON

```
# Old way (4 lines)
nums = []
for x in range(1, 6):
    nums.append(x)

# List comprehension (1 line!)
nums = [x for x in range(1, 6)]
print(nums) # [1, 2, 3, 4, 5]

# Square every number
squares = [x**2 for x in range(1, 6)]
print(squares) # [1, 4, 9, 16, 25]
```

Filtered — keep only matching items

```
# Even numbers from 1-10
evens = [x for x in range(1, 11) if x % 2 == 0]
print(evens) # [2, 4, 6, 8, 10]

# Words longer than 3 chars
words = ["hi", "hello", "hey", "howdy"]
long_words = [w for w in words if len(w) > 3]
print(long_words) # ['hello', 'howdy']
```

if-else — assign one of two values per item

```
# Label each number "Even" or "Odd"
labels = ["Even" if x % 2 == 0 else "Odd" for x in range(1, 7)]
print(labels) # ['Odd', 'Even', 'Odd', 'Even', 'Odd', 'Even']

# Double evens, halve odds
result = [x*2 if x % 2 == 0 else x//2 for x in range(1, 7)]
```

■ *Mental model: [WHAT FOR each ITEM in WHERE IF condition] — read it like English!*

■ JSON Module

JSON (**J**ava**S**cript **O**bject **N**otation) is a lightweight format to exchange data between programs, APIs, and websites. It looks almost identical to a Python dictionary!

```
import json # always import first
```

The 4 Key JSON Functions

Function	Direction	Use for
<code>json.dumps(obj)</code>	Python dict → JSON string	Serialise Python data to send/store
<code>json.loads(str)</code>	JSON string → Python dict	Parse JSON received from API/file
<code>json.dump(obj, f)</code>	Python dict → JSON file	Write JSON directly to a file

🌸 Python Fundamentals 🌸 Part 5

File I/O • Exception Handling • List Comprehensions • JSON

<code>json.load(f)</code>	JSON file → Python dict	Read JSON directly from a file
---------------------------	-------------------------	--------------------------------

dumps() & loads() — working with strings

```
import json

# Python dict → JSON string
data = {"name": "John", "age": 25, "marks": [85, 90, 92]}
json_string = json.dumps(data)
print(json_string)
# Output: {"name": "John", "age": 25, "marks": [85, 90, 92]}

# JSON string → Python dict
json_data = '{"name": "John", "age": 25}'
python_obj = json.loads(json_data)
print(python_obj["name"]) # John
```

dump() & load() — working with files

```
import json

# Write Python dict to JSON file
data = {"name": "Aisha", "city": "Delhi"}
with open("data.json", "w") as f:
    json.dump(data, f, indent=4) # indent=4 makes it readable

# Read JSON file back into Python dict
with open("data.json", "r") as f:
    loaded = json.load(f)
    print(loaded["name"]) # Aisha
```

■ *indent=4 in json.dump() adds pretty formatting to the file — great for readability!*

■ Quick Cheat Sheet

Topic	Key syntax / function	Remember
Open file	<code>open('file', mode)</code>	Default mode is 'r'
Close file	<code>f.close()</code> OR <code>with open()</code>	Always close — prefer with!
Read all	<code>f.read()</code>	Returns one big string
Read line	<code>f.readline()</code>	Call repeatedly for each line
Read lines list	<code>f.readlines()</code>	Returns list of strings
Write string	<code>f.write('text')</code>	Mode must be 'w' or 'a'
Write list	<code>f.writelines([...])</code>	No auto newlines — add <code>\n</code> yourself
Delete file	<code>os.remove('file')</code>	import os first
Try/Except	<code>try: ... except ExType:</code>	Catch specific errors
Else block	<code>else: ...</code>	Runs only if NO exception

🌸 Python Fundamentals 🌸 Part 5

File I/O • Exception Handling • List Comprehensions • JSON

Finally block	finally: ...	ALWAYS runs — perfect for cleanup
List comp basic	[expr for x in iter]	Replace simple for loops
List comp filter	[expr for x in iter if cond]	Keep matching items only
List comp if-else	[a if cond else b for x in iter]	One of two values per item
Py → JSON str	json.dumps(dict)	Serialise to string
JSON str → Py	json.loads(str)	Parse string to dict
Write JSON file	json.dump(dict, file)	Use indent=4 for pretty print
Read JSON file	json.load(file)	Returns Python dict

■ Files = open/read/write/close | Errors = try/except/else/finally | Lists = [expr for x in iter if cond] ■